

MATRICES

1. Edición y generación de matrices

repaso semana 01.

En MATLAB, la estructura de datos fundamental es la matriz. Una matriz puede entenderse como una tabla formada por filas y columnas donde se almacenan valores numéricos organizados de manera ordenada.

En realidad, dentro de MATLAB casi todo se maneja como una matriz. Incluso cuando trabajamos con un solo número, el programa lo trata internamente como una matriz de 1 fila y 1 columna.

Observación importante:

Aunque muchas veces se usan como sinónimos, un arreglo (array) y una matriz no son exactamente lo mismo.

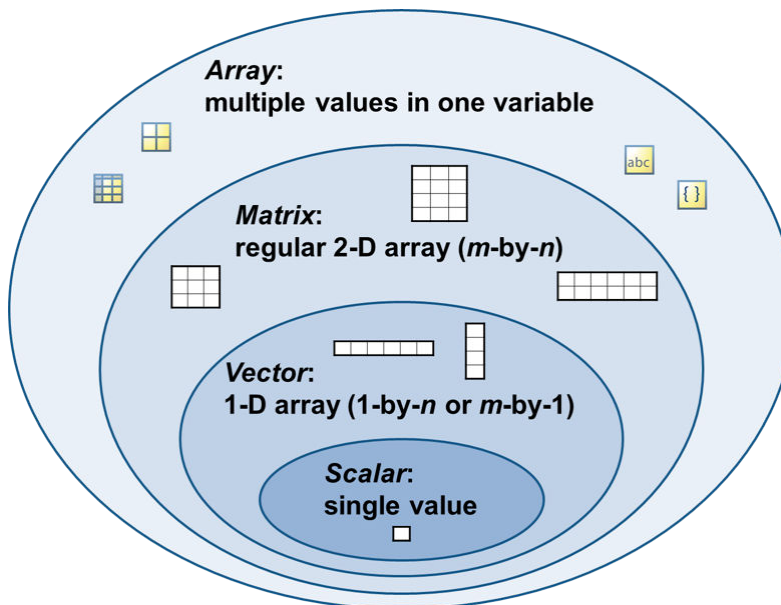
Un arreglo es un concepto más general: es un conjunto organizado de datos que puede tener una o más dimensiones y, dependiendo del lenguaje, puede contener distintos tipos de información.

Una matriz, en cambio, es un tipo específico de arreglo. Sus características principales son:

- Es bidimensional (solo tiene filas y columnas).
- Sus elementos deben ser del mismo tipo de dato.
- Tiene una estructura rectangular bien definida.

En otras palabras, toda matriz es un arreglo, pero no todo arreglo es necesariamente una matriz.

Figura 1. Tipos de arreglos.



Tomado de la documentación oficial de MATLAB.

Diferencia entre escalar, vector y matriz

Dentro de MATLAB es importante distinguir tres conceptos básicos cuando se trabaja con matrices: escalar, vector y matriz. Aunque todos se manejan bajo la misma estructura interna, su diferencia está en sus dimensiones.

Escalar

Un escalar es el caso más simple. Se trata de una matriz que tiene una sola fila y una sola columna, es decir, sus dimensiones son 1×1 . En términos prácticos, es simplemente un número. Por ejemplo:

```
x = 5;
```

Vector

Un vector es un tipo particular de matriz que puede presentarse de dos formas: como vector fila o como vector columna.

Un vector fila tiene dimensiones $1 \times n$, es decir, posee una sola fila y varias columnas.

Un vector columna tiene dimensiones $n \times 1$, lo que significa que tiene varias filas pero una sola columna.

La característica principal que distingue a un vector es que solo tiene una dimensión dominante: o una única fila o una única columna.

```
v_fila = [1 2 3 4];  
  
v_col = [1;  
        2;  
        3;  
        4];
```

Matriz

Para construir una matriz en MATLAB se utilizan símbolos específicos para organizar filas y columnas.

- La coma (,) o un espacio separa las columnas.
- El punto y coma (;) separa las filas.

Esto permite escribir la matriz respetando su estructura rectangular.

Por ejemplo, si queremos definir una matriz de 3 filas y 3 columnas, primero organizamos sus elementos por filas y luego los ingresamos en MATLAB.

Supongamos que tenemos la siguiente matriz:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Ahora creémosla dentro de MATLAB. Esto se realiza a partir de las filas.

```
A = [1 2 3;
     4 5 6;
     7 8 9];
```

Matriz vacía

Antes de continuar es importante conocer el concepto de matriz vacía.

Una matriz vacía es aquella que no contiene ningún elemento. Es decir, existe como estructura, pero no almacena datos. Precisamente por no tener contenido recibe ese nombre.

En MATLAB, la forma más sencilla de crear una matriz vacía es asignando corchetes sin valores:

```
% Matriz vacía genérica
M = [];
M2 = int8.empty;
```

Esta matriz se puede presentar de diferentes formas, no únicamente por dos corchetes sin elementos. Esto dependerá del tipo de dato con el que se esté trabajando, pero es más que suficiente con leer qué tipo de matriz se obtiene. Cuando es una matriz vacía MATLAB suele indicar que se trata de una, además del tipo de matriz.

Notación científica en matrices

Anteriormente vimos cómo expresar números en notación científica dentro de MATLAB cuando trabajábamos con escalares. En ese caso, el número se mostraba directamente en la forma correspondiente (por ejemplo, usando la letra e para indicar la potencia de 10).

Cuando trabajamos con matrices, el concepto es el mismo en cuanto a la representación numérica, pero cambia la manera en que MATLAB muestra los valores en pantalla.

La diferencia principal no está en cómo se escriben los números, sino en cómo se visualizan cuando forman parte de una matriz. MATLAB puede mostrar los elementos utilizando notación científica automáticamente, especialmente cuando los valores son muy grandes o muy pequeños.

```
A = [1 2;
     3 6e10]
```

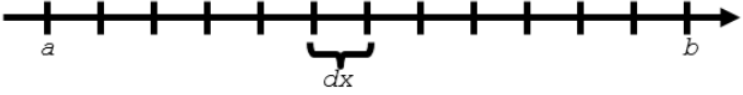
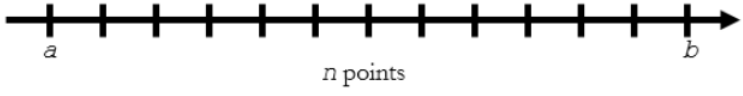
```
A = 2x2
10^10 ×
    0.0000    0.0000
    0.0000    6.0000
```

Por defecto, la notación científica de una matriz siempre se visualizará cuando al menos uno de los elementos de ésta sea muy grande o muy pequeño.

Crear vectores con el operador dos puntos y la función linspace

Es posible crear vectores **linealmente espaciados** con el *operador dos puntos*, `:`, o con la función `linspace`. Sin embargo, se debe diferenciar cuánto utilizar un método u otro.

Figura 2. Operador dos puntos VS. Función linspace.

Syntax	Vector Created	When to use
<code>x = a:dx:b</code>		When element spacing dx is known.
<code>x = linspace(a,b,n)</code>		When number of elements n is known.

Tomado de la documentación oficial de MATLAB.

Operador dos puntos :

El operador dos puntos : permite crear vectores linealmente espaciados conociendo:

- el número inicial
- el número final
- el espaciamiento

```
x = 3 :3 :30;
```

Función linspace

La función `linspace` permite crear vectores linealmente espaciados conociendo:

- el número inicial
- el número final
- el número de elementos

```
x = linspace(3, 30, 10);
```

Observaciones

- Si no se indica el número de elementos, MATLAB que es 100.

Función logspace

La función **logspace** se utiliza para generar vectores cuyos valores están distribuidos de manera logarítmica en base 10.

A diferencia de otros comandos donde el incremento entre elementos es constante, en este caso los valores crecen (o decrecen) siguiendo potencias de 10.

Para usar esta función es necesario conocer tres datos:

- La potencia del número inicial
- La potencia del número final
- La cantidad total de elementos que se desean generar

Es decir, si se quieren obtener varios números distribuidos logarítmicamente desde un valor inicial hasta un valor final, no se ingresan directamente esos números, sino las potencias de 10 que los representan.

sintaxis

Variable = logspace(potencialInicial, potenciaFinal, numeroDeElementos)

```
x = logspace(0, 3, 4);
```

Observación

- Si no se indica el número de elementos, MATLAB que es 50.

2. Dimensión

Dimensión predominante en una matriz

Cuando se trabaja con matrices en MATLAB es importante comprender cómo el programa interpreta las dimensiones.

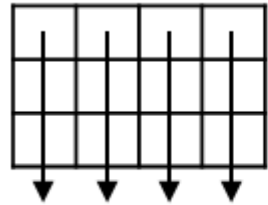
En MATLAB, la dimensión que se considera primero al aplicar muchas funciones es la columna, también conocida como dimensión 1. Después se encuentra la fila, llamada dimensión 2.

Esto significa que, por defecto, varias funciones operan recorriendo las columnas antes que las filas.

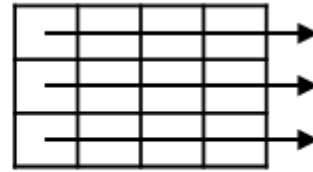
Cuando se trabaja únicamente con vectores, esta diferencia casi no se percibe. Sin embargo, al trabajar con matrices de varias filas y columnas, sí es fundamental tenerlo en cuenta, porque afecta directamente el resultado de ciertas funciones.

Por ejemplo, analicemos qué ocurre con la función sum.

Figura 3. Dimensión predominante y dimensión secundaria de una matriz en MATLAB.



Dimension = 1



Dimension = 2

Tomado de la documentación oficial de MATLAB.

Por ejemplo, veamos qué ocurre con la función `sum` (suma) cuando se aplica a vectores y matrices.

```
A = [1 2 3;
      4 5 6;
      7 8 9];
```

```
sum(A, 2)
```

```
ans = 3x1
      6
     15
     24
```

Observación

En MATLAB, como se mencionó anteriormente, la mayoría de las funciones consideran las columnas como dimensión principal al realizar operaciones. Es decir, si no se especifica nada adicional, el cálculo se aplica recorriendo la dimensión 1.

No obstante, existen funciones que no siguen este comportamiento por defecto y permiten trabajar directamente sobre las filas o modificar explícitamente la dimensión de operación. Unas de estas funciones es `size` y `circshift`.

Determinar las dimensiones y el número de elementos

Cuando tenemos una matriz es importante saber cuáles son sus dimensiones, además de cuántos elementos hay en esta matriz. A continuación se presentan las funciones para este fin.

Funciones `size` y `length`

La función `size` permite conocer las dimensiones de una matriz.

```
% size
sizeMatrix = [1 2 3; 1 2 3; 1 2 3; 1 2 3]
```

```
sizeMatrix = 4x3
      1      2      3
      1      2      3
      1      2      3
      1      2      3
```

```
sizeVector = [1 2 3]
```

```
sizeVector = 1×3  
1      2      3
```

```
size(sizeMatrix, 1)
```

```
ans =  
4
```

El primer número representa el número de filas y el segundo número representa el número de columnas.

Por otra parte, la función `length` devuelve el valor de la dimensión más grande.

```
length(sizeMatrix)
```

```
ans =  
4
```

La función `length` es especialmente útil cuando se trabajan con vectores, puesto que la dimensión más grande coincide con el número de elementos del vector. No obstante, cuando se quiere conocer cuál es el valor de la dimensión más grande únicamente, también resulta bastante útil.

Función `numel`

La función `numel` permite conocer el número de elementos dentro de una matriz.

```
numel(a)
```

```
ans =  
9
```

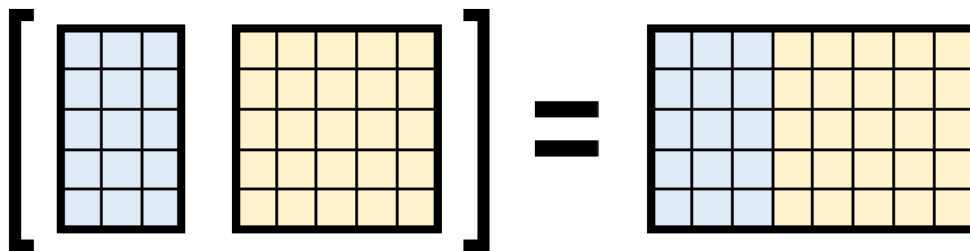
3. Redimensionamiento de matrices

Concatenar matrices

Teniendo en cuenta las dimensiones de las matrices, es posible concatenar (unir) una o más matrices siempre y cuando el resultado sea una matriz $m \times n$.

Para concatenar matrices también se utilizan corchetes `[]`.

Figura 4. Concatenación horizontal.



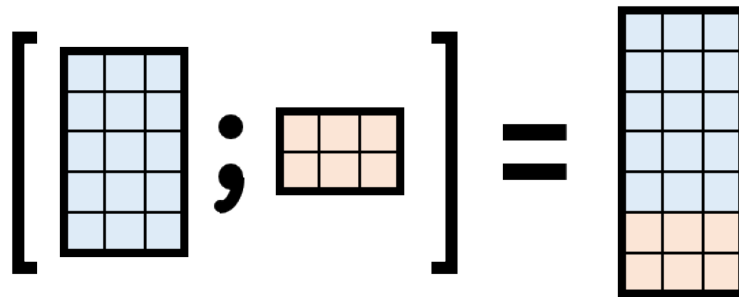
```
a = [ 1 2 3;  
      3 4 5];
```

```
b = [6 9 ;  
     7 8];
```

```
c = [a b]
```

```
c = 2x5  
     1     2     3     6     9  
     3     4     5     7     8
```

Figura 5. Concatenación vertical.



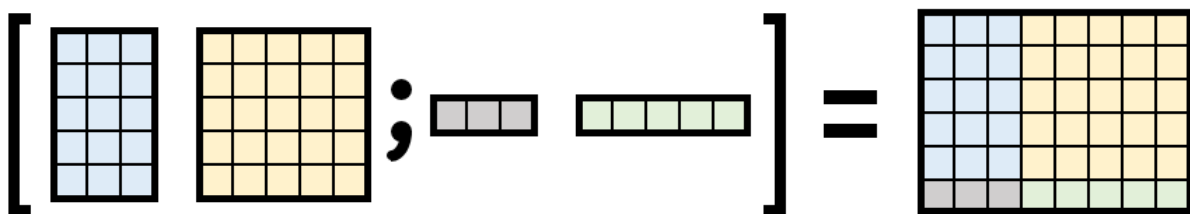
```
c = [ 1 2 3;  
     3 4 5];
```

```
d = [6 9 4 ;  
     7 8 4];
```

```
e = [c ; d]
```

```
e = 4x3  
     1     2     3  
     3     4     5  
     6     9     4  
     7     8     4
```

Figura 6. Concatenación combinada.



Todas las figuras fueron tomadas de de la documentación oficial de MATLAB.

```
f = [1 ;  
     2];
```

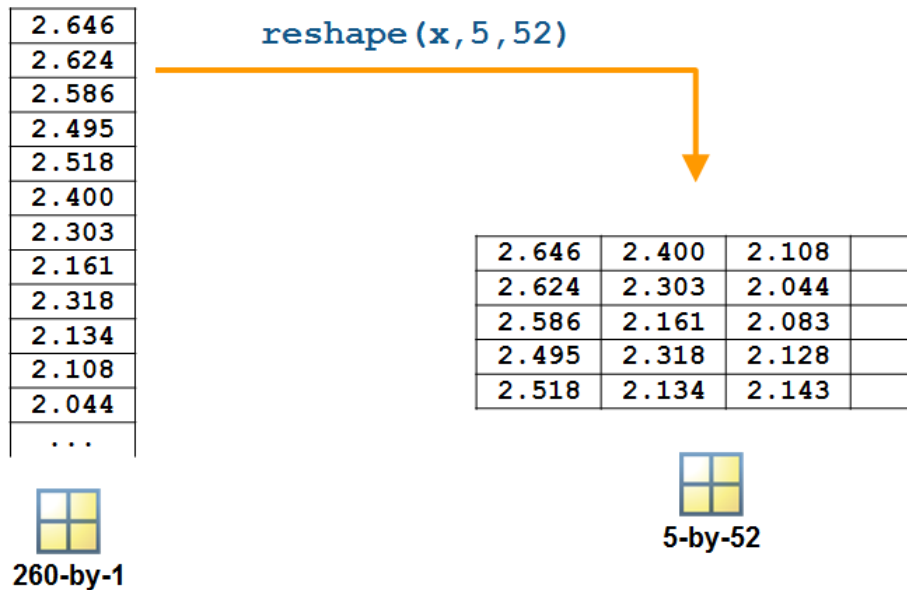
```
g = [b f; c]
```

```
g = 4x3  
     6     9     1  
     7     8     2  
     1     2     3  
     3     4     5
```


Redimensionar matrices

En MATLAB existen funciones que permiten reorganizar los elementos de una matriz sin cambiar sus valores. Es decir, los datos siguen siendo los mismos, pero se colocan en otra forma o secuencia.

Figura 7. Redimensionamiento de una matriz.



Tomado de la documentación oficial de MATLAB.

Función reshape

La función `reshape` permite redimensionar una matriz.

Lo que se debe considerar es que la nueva matriz redimensionada debe contener a todos los elementos de la matriz original. Es decir, el producto de las dimensiones de cualquier matriz redimensionada debe ser igual al producto de las dimensiones de la matriz original.

% Sintaxis

`MatrizRedimensionada = reshape(MatrizOriginal, Dimensiones)`

Por ejemplo, en lugar de tener una matriz 6×4 se podría redimensionar a una matriz 8×3 .

```
a = randi([0 10], 35,1)
```

```
a = 35x1
    7
    1
    6
    2
    9
    9
    4
    4
   10
    0
    4
    5
```

```
6
0
1
:
:
```

Observación

En realidad, si el arreglo es bidimensional (una matriz), es más que suficiente especificarle una dimensión a la función `reshape` para que ésta infiera la otra dimensión. Lo que se debe tener en cuenta es que la dimensión especificada tiene que ser un múltiplo del producto de las dimensiones de la matriz original.

```
a = reshape(a,7, 5)
```

```
a = 7x5
    7     4     1     9     3
    1    10     7     8     3
    6     0     2     1    10
    2     4     1     0     4
    9     5     0     7     8
    9     6     1     5     2
    4     0     5     5     0
```

Funciones `flipud` y `fliplr`

La función `flipud` permite invertir una matriz verticalmente, es decir, cambia el orden de las filas de arriba hacia abajo (**flip up to down**). Por otro lado, la función `fliplr` invierte la matriz horizontalmente, cambiando el orden de las columnas de izquierda a derecha (**flip left to right**).

```
fliplr(a)
```

```
ans = 7x5
     3     9     1     4     7
     3     8     7    10     1
    10     1     2     0     6
     4     0     1     4     2
     8     7     0     5     9
     2     5     1     6     9
     0     5     5     0     4
```

```
flipud(a)
```

```
ans = 7x5
     4     0     5     5     0
     9     6     1     5     2
     9     5     0     7     8
     2     4     1     0     4
     6     0     2     1    10
     1    10     7     8     3
     7     4     1     9     3
```

Ordenar matrices

Ordenar los datos en una matriz también es una herramienta valiosa y MATLAB ofrece varios enfoques.

Función sort

Perfecto 💎 te lo dejo bien redactado, claro y formal para tu material:

Función sort

La función `sort` permite ordenar los elementos de una matriz, ya sea por filas o por columnas, de manera independiente y en orden ascendente o descendente.

- Si la matriz es un **vector**, `sort` ordena todos sus elementos.
- Si la matriz es de tamaño **m × n**, la función considera cada columna como un vector y ordena cada una por separado (comportamiento por defecto).

Además, se pueden especificar dos aspectos importantes:

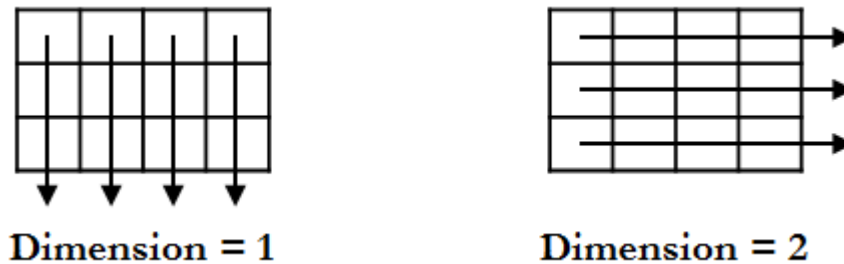
• Dimensión

- 1 → ordenar por columnas (valor por defecto)
- 2 → ordenar por filas

• Dirección

- 'ascend' → orden ascendente (valor por defecto)
- 'descend' → orden descendente

Figura 8. Dimensiones de una matriz dentro de una función interna.



Tomado de la documentación oficial de MATLAB.

```
sort(a, 1, "ascend")
```

```
ans = 7x5
     1     0     0     0     0
     2     0     1     1     2
     4     4     1     5     3
     6     4     1     5     3
     7     5     2     7     4
     9     6     5     8     8
     9    10     7     9    10
```

Función sortrows

La función `sortrows` permite ordenar filas o columnas enteras entre sí, en función de una columna de referencia.

Notemos que este es un inconveniente con la anterior función `sort`, puesto que las columnas o filas son tratadas de forma independiente. Si no queremos que se pierda la relación entre fila y columna, es preferible utilizar esta función `sortrows`.

También se puede indicar a la función `sortrows`:

- **La columna de referencia:** depende de cuántas columnas tenga la matriz, pero si no se indica entonces se tomará como referencia la primera columna.
- **La dirección:** 'ascend' (por defecto) o 'descend'.

```
sortrows(a,1,"descend")
```

```
ans = 7x5
     9     5     0     7     8
     9     6     1     5     2
     7     4     1     9     3
     6     0     2     1    10
     4     0     5     5     0
     2     4     1     0     4
     1    10     7     8     3
```

```
a
```

```
a = 7x5
     7     4     1     9     3
     1    10     7     8     3
     6     0     2     1    10
     2     4     1     0     4
     9     5     0     7     8
     9     6     1     5     2
     4     0     5     5     0
```

4. Subíndices de vectores y matrices

Indexación de matrices

Indexar significa acceder a los elementos de una matriz utilizando su posición dentro de ella.

En MATLAB existen tres formas principales de acceder a los datos según su ubicación (índice):

- Indexación por posición
- Indexación lineal
- Indexación lógica

Cada una permite trabajar con los elementos de manera distinta, dependiendo de lo que se necesite hacer.

Indexación por posición

Analicemos la siguiente matriz.

$$A = \begin{bmatrix} 4_{1,1} & 8_{1,2} & 0_{1,3} \\ 9_{2,1} & 2_{2,2} & 1_{2,3} \\ 7_{3,1} & 5_{3,2} & 6_{3,3} \end{bmatrix}$$

Si observamos una matriz, cada elemento tiene una posición específica identificada por índices, también llamados subíndices.

Al igual que en la notación matemática tradicional, el primer índice indica la fila y el segundo índice indica la columna.

En MATLAB es posible realizar distintos tipos de indexación:

- **Un solo elemento:** Se especifica la fila y la columna exacta del elemento que se desea acceder.
- **Submatrices:** Se indican los índices de las filas y columnas que se desean extraer.
- **Filas completas:** Se escribe el número de la fila y, para las columnas, se utiliza el operador dos puntos :.
- **Columnas completas:** Se escribe el número de la columna y, para las filas, se utiliza el operador :.

El operador dos puntos : como índice significa “todos los elementos” en esa dimensión.

Además, este operador permite indicar un rango de índices, por ejemplo 1:3, lo que evita escribir cada posición manualmente.

```
a
```

```
a = 7x5
```

```

7      4      1      9      3
1     10      7      8      3
6      0      2      1     10
2      4      1      0      4
9      5      0      7      8
9      6      1      5      2
4      0      5      5      0
```

```
% un unico elemento
```

```
a(1,2)
```

```
ans =
```

```
4
```

```
%fila completa
```

```
a(2,:)
```

```
ans = 1x5
```

```
1     10      7      8      3
```

```
%columna completa
```

```
a(:,1)
```

```
ans = 7×1
7
1
6
2
9
9
4
```

Palabra reservada end

La palabra reservada end en indexación permite hacer referencia al último elemento de una matriz. En síntesis, es un número que hace referencia al último elemento de una matriz, independientemente del tamaño de la matriz o de si ésta cambia sus dimensiones.

```
a
```

```
a = 7×5
7    4    1    9    3
1   10    7    8    3
6    0    2    1   10
2    4    1    0    4
9    5    0    7    8
9    6    1    5    2
4    0    5    5    0
```

```
a(end,1)
```

```
ans =
4
```

Se tendría que seguir la misma lógica para obtener algún elemento empezando desde el final.

Convertir una matriz en un vector

En el caso de que se necesite convertir una matriz a un vector, se podría indexar fila a fila, o columna a columna, e ir concatenando estas filas, o columnas, para formar el vector. No obstante, el operador dos puntos : facilita este trabajo.

Para convertir una matriz en un vector se debe indexar la matriz únicamente con el operador dos puntos :.

```
a(:)
```

```
ans = 35×1
7
1
6
2
9
9
4
4
4
10
0
4
5
6
```

0
1
⋮

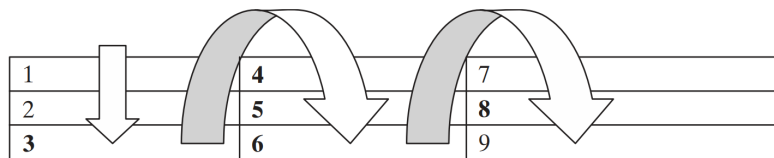
Indexación lineal o sencilla

La **indexación lineal**, también llamada indexación sencilla, consiste en acceder a los elementos de una matriz utilizando un solo índice en lugar de indicar fila y columna.

En este caso surge la pregunta: ¿cómo sabe MATLAB qué elemento corresponde a ese único índice?

MATLAB recorre la matriz por columnas, no por filas. Es decir, primero toma todos los elementos de la primera columna (de arriba hacia abajo), luego continúa con la segunda columna, y así sucesivamente.

Figura 9. Proceso de indexación sencilla.



Tomado de MATLAB para ingenieros de Holly Moore.

En la figura anterior se puede observar cómo se asignan los índices lineales dentro de una matriz. Este mismo criterio es el que utiliza MATLAB cuando se transforma una matriz en un vector usando la notación $A(:)$.

Analicemos la misma matriz anterior con sus índices sencillos:

$$A = \begin{bmatrix} 4_1 & 8_4 & 0_7 \\ 9_2 & 2_5 & 1_8 \\ 7_3 & 5_6 & 6_9 \end{bmatrix}$$

Con esta indexación únicamente podemos extraer:

- Elementos únicos.
- Vectores fila de elementos indicando los índices sencillos.
- Vectores fila de elementos utilizando el operador dos puntos.

```
a(end)
```

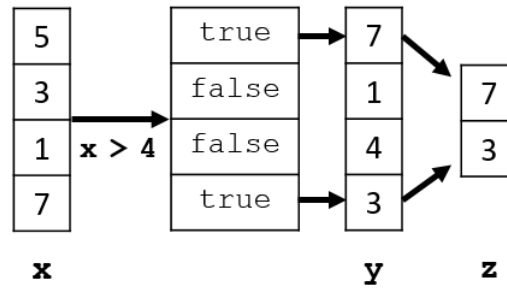
```
ans =  
0
```

Esta indexación es especialmente útil con vectores en lugar de matrices.

Indexación lógica

Para el análisis de la indexación lógica se tiene la siguiente figura.

Figura 10. Conversión de subíndices a índices.



Tomado de la documentación oficial de MATLAB.

El uso de valores lógicos es otra forma útil de indexar matrices, especialmente cuando se trabaja con declaraciones condicionales porque se establece una condición y se indexan únicamente los valores que la cumplen.

```
b = [5;3;1;7]
```

```
b = 4x1
     5
     3
     1
     7
```

```
b(b>3)
```

```
ans = 2x1
     5
     7
```

Reemplazar elementos de una matriz

Utilizando cualquier tipo de indexación se puede reemplazar elementos de una matriz. Únicamente se indexan los elementos y se asignan los nuevos valores.

```
a(1, 4) = 4
```

```
a = 7x5
     7     4     1     4     3
     1    10     7     8     3
     6     0     2     1    10
     2     4     1     0     4
     9     5     0     7     8
     9     6     1     5     2
     4     0     5     5     0
```

Esto funciona tanto para reemplazar un único elemento o varios elementos. Lo que se debe considerar es que la matriz que se indexa debe ser del mismo tamaño (tener las mismas dimensiones) que la matriz que reemplazará a la matriz indexada.

```
a(1:2, 1:2) = [1, 2; 1 2]
```

```
a = 7x5
     1     2     1     4     3
     1     2     7     8     3
```


6	0	2	1	10
2	4	1	0	4
9	5	0	7	8
9	6	1	5	2
4	0	5	5	0

Expandir una matriz

Una forma de expandir una matriz es concatenarla con otros elementos o matrices. No obstante, mediante indexación también es posible expandir una matriz.

Se debe indicar un índice fuera de rango de la matriz, es decir, que sea más grande que las dimensiones de la matriz y asignarle un valor o valores. Esto forzará a la matriz a expandirse rellenando los elementos en las posiciones que no se hayan asignado nada con ceros.

Lo que siempre se debe tener en cuenta es que la matriz resultante siempre debe ser consistente, dimensionalmente hablando.

```
a = [1, 2 ; 3, 4]
```

```
a = 2x2
     1     2
     3     4
```

```
b = [1, 2 ; 3, 4];
a(8:9,8:9) = b
```

```
a = 9x9
     1     2     0     0     0     0     0     0     0
     3     4     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     1     2
     0     0     0     0     0     0     0     3     4
```

5. Borrando filas o columnas.

Eliminar elementos de una matriz

Para eliminar elementos de una matriz se utiliza la matriz vacía []. Basta con utilizar cualquier indexación e indexar los elementos que se desean eliminar y asignarles la matriz vacía.

```
a(:,3) = []
```

```
a = 9x8
     1     2     0     0     0     0     0     0
     3     4     0     0     0     0     0     0
     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     1     2
```

6. Operaciones con matrices

En MATLAB existen dos formas principales de realizar operaciones matemáticas: las operaciones matriciales y las operaciones arreglo. Ambas permiten efectuar cálculos numéricos como sumas, potencias o multiplicaciones, pero funcionan de manera diferente.

Operaciones arreglo (elemento a elemento)

Las operaciones arreglo trabajan comparando los elementos que ocupan la misma posición dentro de las matrices o vectores. Es decir, el cálculo se hace elemento por elemento, no considerando la estructura algebraica de la matriz como en la multiplicación matricial tradicional.

Para que este tipo de operación pueda realizarse correctamente deben cumplirse ciertas condiciones:

- Si las matrices tienen exactamente las mismas dimensiones, cada elemento de una se combina con el elemento que está en la misma fila y columna de la otra.
- Si las dimensiones no son idénticas pero son compatibles, MATLAB ajusta automáticamente (expansión implícita) una de las matrices para que coincida con la otra y así pueda ejecutarse la operación.
- Si las dimensiones no coinciden ni son compatibles, MATLAB no podrá efectuar el cálculo y mostrará un mensaje de error.

Estas son algunas combinaciones de escalares, vectores y matrices que tienen tamaños compatibles:

Figura 1. Operación arreglo entre dos matrices que tienen el mismo tamaño.

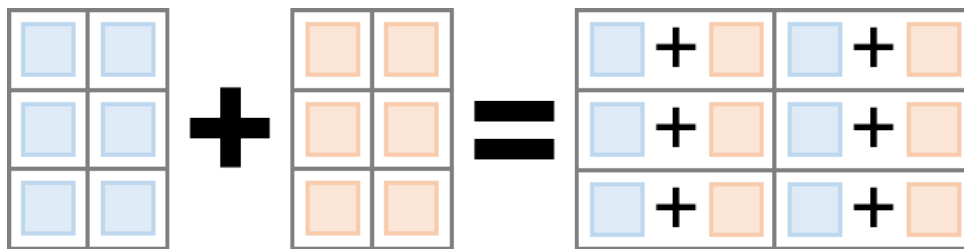


Figura 2. Operación arreglo entre una matriz y un escalar.

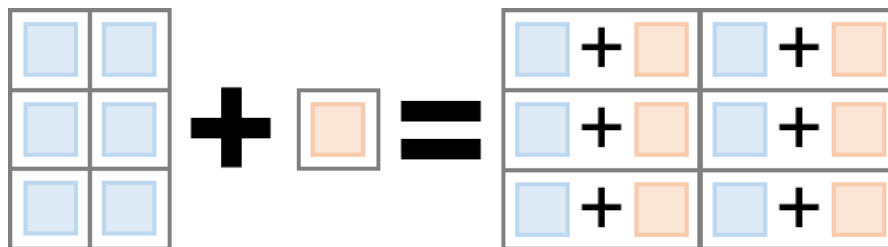


Figura 3. Operación arreglo entre una matriz y un vector fila. Ambos tienen el mismo número de columnas.

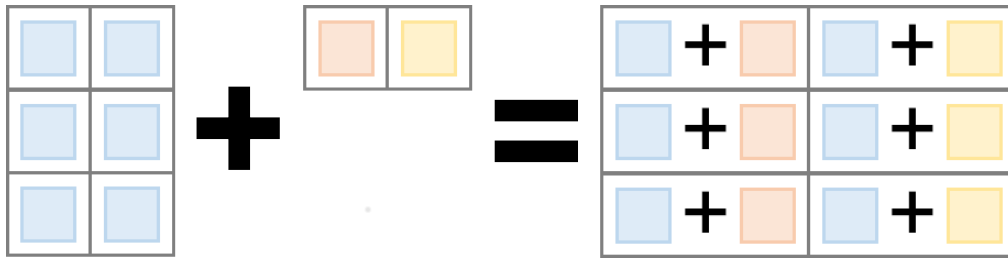
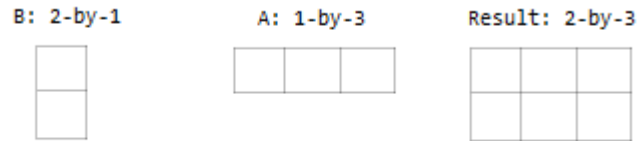


Figura 4. Operación arreglo entre un vector columna y un vector fila.



Operaciones elementales

Previamente se estudió las operaciones con escalares. Ahora se expandirá lo explicado anteriormente a operaciones elementales con matrices.

Figura 5. Todas las operaciones arreglo dentro de MATLAB.

Operator	Purpose	Description	Reference Page
+	Addition	A+B adds A and B.	plus
+	Unary plus	+A returns A.	uplus
-	Subtraction	A-B subtracts B from A	minus
-	Unary minus	-A negates the elements of A.	uminus
.*	Element-wise multiplication	A.*B is the element-by-element product of A and B.	times
.^	Element-wise power	A.^B is the matrix with elements A(i,j) to the B(i,j) power.	power
./	Right array division	A./B is the matrix with elements A(i,j)/B(i,j).	rdivide
.\	Left array division	A.\B is the matrix with elements B(i,j)/A(i,j).	ldivide
.'	Array transpose	A.' is the array transpose of A. For complex matrices, this does not involve conjugation.	transpose

Tomado de la documentación oficial de MATLAB.

```
% matrices
```

```
A = [-1 2 3;  
      4 -5 6;  
      7 8 -9];
```

$$B = \begin{bmatrix} 9, & 4, & 3; \\ 8, & 5, & 2; \\ 7, & 6, & 1 \end{bmatrix};$$

$$C = \begin{bmatrix} 2 & 4; \\ 6 & 8 \end{bmatrix};$$

$$V1 = \begin{bmatrix} 2; \\ 4 \end{bmatrix};$$

$$V2 = [2, 3, 4];$$

suma (+)

$$R1 = A$$

$$R1 = \begin{matrix} 3 \times 3 \\ \begin{bmatrix} -1 & 2 & 3 \\ 4 & -5 & 6 \\ 7 & 8 & -9 \end{bmatrix} \end{matrix}$$

$$R2 = A + V2$$

$$R2 = \begin{matrix} 3 \times 3 \\ \begin{bmatrix} 1 & 5 & 7 \\ 6 & -2 & 10 \\ 9 & 11 & -5 \end{bmatrix} \end{matrix}$$

$$R3 = V1 + V2$$

$$R3 = \begin{matrix} 2 \times 3 \\ \begin{bmatrix} 4 & 5 & 6 \\ 6 & 7 & 8 \end{bmatrix} \end{matrix}$$

resta (-)

$$R1 = A - B$$

$$R1 = \begin{matrix} 3 \times 3 \\ \begin{bmatrix} -10 & -2 & 0 \\ -4 & -10 & 4 \\ 0 & 2 & -10 \end{bmatrix} \end{matrix}$$

$$R2 = A - V2$$

$$R2 = \begin{matrix} 3 \times 3 \\ \begin{bmatrix} -3 & -1 & -1 \\ 2 & -8 & 2 \\ 5 & 5 & -13 \end{bmatrix} \end{matrix}$$

$$R3 = V1 - V2$$

$$R3 = \begin{matrix} 2 \times 3 \\ \begin{bmatrix} 0 & -1 & -2 \end{bmatrix} \end{matrix}$$

2 1 0

Multiplicación (-)

$$R1 = A .* B$$

$$R1 = 3 \times 3$$

-9	8	9
32	-25	12
49	48	-9

$$R2 = A .* V2$$

$$R2 = 3 \times 3$$

-2	6	12
8	-15	24
14	24	-36

$$R3 = V1 .* V2$$

$$R3 = 2 \times 3$$

4	6	8
8	12	16

división (-)

$$R1 = A ./ B$$

$$R1 = 3 \times 3$$

-0.1111	0.5000	1.0000
0.5000	-1.0000	3.0000
1.0000	1.3333	-9.0000

$$R2 = A .\ V2$$

$$R2 = 3 \times 3$$

-2.0000	1.5000	1.3333
0.5000	-0.6000	0.6667
0.2857	0.3750	-0.4444

potencia (-)

$$R1 = A .^ B$$

$$R1 = 3 \times 3$$

-1	16	27
65536	-3125	36
823543	262144	-9

$$R2 = A .^ V2$$

$$R2 = 3 \times 3$$

1	8	81
16	-125	1296
49	512	6561

$$R3 = V1 .^ V2$$

$$R3 = 2 \times 3$$

4	8	16
---	---	----

Transpuesta (')

```
T1 = A'
```

```
T1 = 3x3
    -1     4     7
     2    -5     8
     3     6    -9
```

```
T2 = C.'
```

```
T2 = 2x2
     2     6
     4     8
```

```
T3 = V1.'
```

```
T3 = 1x2
     2     4
```

Operaciones matriciales

Las operaciones matriciales dentro de MATLAB siguen las **reglas del álgebra lineal** que se estudian en el instituto y en la universidad.

Operaciones matriciales

En la siguiente tabla se muestran las operaciones matriciales de las que se dispone dentro de MATLAB.

Figura 1. Todas las operaciones matriciales dentro de MATLAB.

Operator	Purpose	Description	Reference Page
	Matrix multiplication	$C = A*B$ is the linear algebraic product of the matrices A and B. The number of columns of A must equal the number of rows of B.	mtimes
	Matrix left division	$x = A \setminus B$ is the solution to the equation $Ax = B$. Matrices A and B must have the same number of rows.	mldivide
	Matrix right division	$x = B/A$ is the solution to the equation $xA = B$. Matrices A and B must have the same number of columns. In terms of the left division operator, $B/A = (A' \setminus B')'$.	mrdivide
	Matrix power	A^B is A to the power B, if B is a scalar. For other values of B, the calculation involves eigenvalues and eigenvectors.	mpower
	Complex conjugate transpose	A' is the linear algebraic transpose of A. For complex matrices, this is the complex conjugate transpose.	ctranspose

Tomado de la documentación oficial de MATLAB.

La suma y resta matricial son similares a la suma y resta tradicional (*operación elemento a elemento*). El resto de operaciones sí cambian significativamente.

Multiplicación y potencia

El producto o multiplicación matricial se define como:

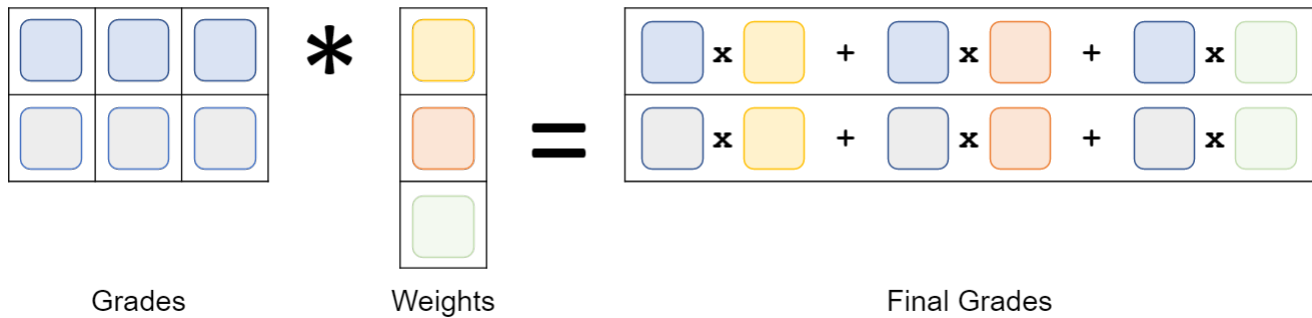
$$AB = (a_{ij})_{m \times n} (b_{ij})_{n \times p} = \left(\sum_{k=1}^n a_{ik} b_{kj} \right)_{m \times p}$$

O escrito de otra forma:

$$AB = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1p} \\ b_{21} & b_{22} & \dots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \dots & b_{np} \end{bmatrix}$$

$$= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + \dots + a_{1n}b_{n1} & \sum_{k=1}^n a_{1k}b_{k2} & \dots & \sum_{k=1}^n a_{1k}b_{kp} \\ a_{21}b_{11} + a_{22}b_{21} + \dots + a_{2n}b_{n1} & \sum_{k=1}^n a_{2k}b_{k2} & \dots & \sum_{k=1}^n a_{2k}b_{kp} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{11} + a_{m2}b_{21} + \dots + a_{mn}b_{n1} & \sum_{k=1}^n a_{mk}b_{k2} & \dots & \sum_{k=1}^n a_{mk}b_{kp} \end{bmatrix}$$

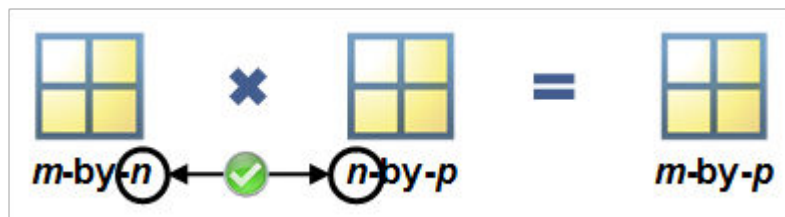
Figura 2. Ejemplo de multiplicación matricial con calificaciones y ponderaciones.



Tomado de la documentación oficial de MATLAB.

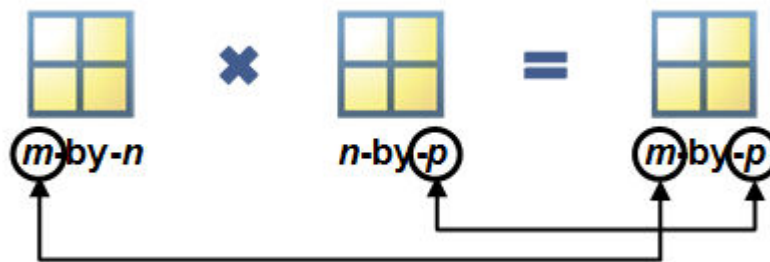
Un pequeño tip para verificar si una matriz se puede multiplicar con otra es observar el número de columnas de la matriz de la derecha y el número de filas de la matriz de la izquierda. Si estos números son iguales, entonces la multiplicación entre estas matrices es posible. Recordar que el producto matricial no es conmutativo, por ende, $AB \neq BA$.

Figura 3. Verificación de dimensiones para realizar la multiplicación matricial.



Tomado de la documentación oficial de MATLAB.

Figura 4. Matriz resultante de la multiplicación matricial.



Tomado de la documentación oficial de MATLAB.

```
sucursalesProductos = randi(10, 5,3)
```

```
sucursalesProductos = 5x3
```

```
3    2    4
6    5    7
7    4    6
10   1    1
1    9    7
```

```
% precios = randi(10,3,1);
%
% totalPagar = sucursalesProductos * precios
```

Por otra parte, la potencia de una matriz es posible únicamente para matrices cuadradas.

```
potencia = randi(10, 3,3);
```

```
potencia ^ 2;
```

Ejercicio

Usted y un amigo van a una tienda. Sus listas son las que se muestran en la tabla.

Figura 5. Cantidad de artículos comprados por cada persona.

Artículo	Cantidad que necesita Ann	Cantidad que necesita Fred
Leche	2 galones	3 galones
Huevos	1 docena	2 docenas
Cereal	2 cajas	1 caja
Sopa	5 latas	4 latas
Galletas	1 paquete	3 paquetes

Tomado de MATLAB para ingenieros de Holly Moore.

Los artículos tienen los siguientes costos:

Figura 6. Precio unitario de cada artículo.

Artículo	Costo
Leche	\$3.50 por galón
Huevos	\$1.25 por docena
Cereal	\$4.25 por caja
Sopa	\$1.55 por lata
Galletas	\$3.15 por paquete

Tomado de MATLAB para ingenieros de Holly Moore.

Encuentre la factura total para cada comprador.

Ejercicio tomado de Moore, 2007.

Solucion

```
Cantidades = [2 1 2 5 1; % Ann
              3 2 1 4 3] % Fred
```

```
Cantidades = 2x5
    2    1    2    5    1
    3    2    1    4    3
```

```
% Costos unitarios
```

```
Costo = [3.50 1.25 4.25 1.55 3.15]'
```

```
Costo = 5x1
    3.5000
    1.2500
    4.2500
    1.5500
    3.1500
```

```
Total = Cantidades * Costo
```

```
Total = 2x1
27.6500
32.9000
```

División izquierda, división derecha

Las divisiones tanto izquierda como derecha dentro de MATLAB permiten resolver un sistema de ecuaciones lineales.

- La división izquierda ($\backslash$) permite resolver el sistema $Ax = b$, donde $x = A \backslash b$. La matriz A y el vector b deben tener el mismo número de filas. El sistema ingresado debe ser similar al mostrado a continuación:

$$a_1x_1 + a_2x_2 + a_3x_3 = b_1$$

$$a_4x_1 + a_5x_2 + a_6x_3 = b_2$$

$$a_7x_1 + a_8x_2 + a_9x_3 = b_3$$

$$A = \begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix}, x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- La división derecha ($/$) permite resolver el sistema $xA = b$, donde $x = b/A$. La matriz A y el vector b deben tener el mismo número de columnas. El sistema ingresado debe ser similar al mostrado a continuación:

$$a_1x_1 + a_2x_2 + a_3x_3 = b_1$$

$$a_4x_1 + a_5x_2 + a_6x_3 = b_2$$

$$a_7x_1 + a_8x_2 + a_9x_3 = b_3$$

$$A = \begin{bmatrix} a_1 & a_4 & a_7 \\ a_2 & a_5 & a_8 \\ a_3 & a_6 & a_9 \end{bmatrix}, x = [x_1 \ x_2 \ x_3], b = [b_1 \ b_2 \ b_3]$$

Donde A es la matriz de coeficientes, b es el vector columna de términos independientes y x es el vector columna solución.

```
datos = 3;
a = randi([-10 10], datos)
```

```
a = 3x3
    7    -2     3
    4    -9    -6
    2    10    -9
```

```
b = randi([-10 10], datos, 1)
```

```
b = 3x1
   -5
    8
   -6
```

```
% x = br1 / ar1  
x = a \ b;
```

Gráficas en 2-D (bidimensionales)

Las gráficas bidimensionales (2-D) más básicas en MATLAB se generan a partir de uno o dos vectores.

Si se utiliza únicamente un vector *y*, MATLAB toma automáticamente los índices del vector como valores del eje horizontal (*x*). Es decir, los puntos del eje *x* serán 1, 2, 3, ... según la posición de cada elemento.

Si se utilizan dos vectores *x* y *y*, entonces:

- El vector *x* representa el eje horizontal.
- El vector *y* representa el eje vertical.

En este caso, ambos vectores deben tener la misma cantidad de elementos.

Función `plot`

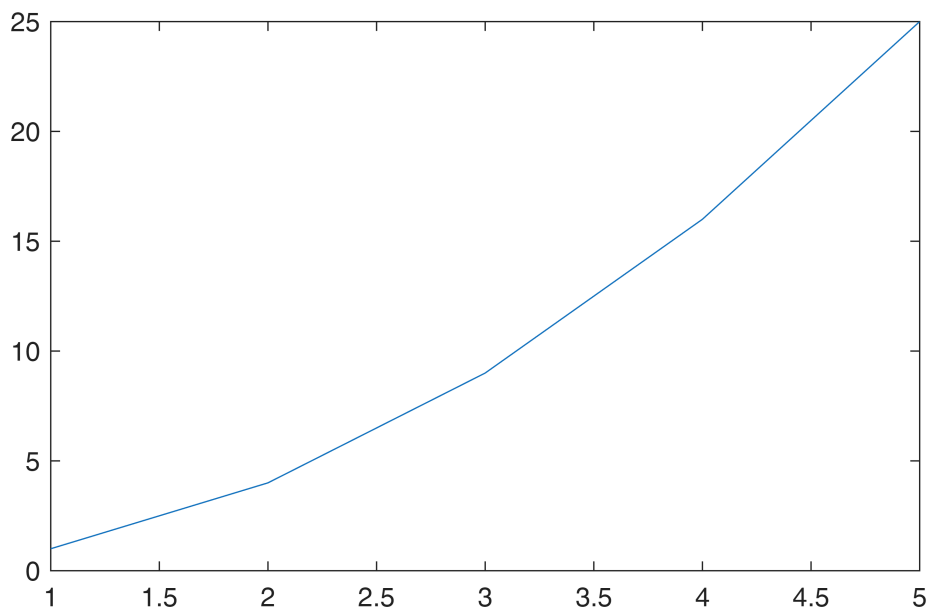
La función `plot` se utiliza para generar gráficas de línea en dos dimensiones.

Permite representar la relación entre dos conjuntos de datos, donde uno corresponde al eje horizontal (*x*) y el otro al eje vertical (*y*).

Sintaxis

Forma básica: `plot(x, y)`

```
x = [1 2 3 4 5];  
y = x .^ 2;  
plot(x , y)
```



```
x2 = linspace(0,2*pi);  
y2 = sin(x2);  
plot(x2,y2)
```

